Which wrappers are supported on any particular PHP installation depends on a number of factors, such as installed extensions or compile-time options for PHP binaries. Full details can be found at https://www.php.net/manual/en/wrappers.php. The php:// wrapper provides access to various I/O streams such as the Unix standard streams like STDOUT ("STandarD OUTput"), the 'body' data of HTTP requests, direct access to file descriptors (the operating system's way of representing files and network connections), and even memory/disk-backed temporary storage. Here's a summary of the php:// wrappers.

Table 1-9. *PHP wrapper I/O streams*

php://stdin php://stdout php://stderr	Unix standard streams https://en.wikipedia.org/wiki/Standard_streams
php://memory php://temp	Temporary store either in memory or a temp file.
php://input	Read-only access to raw data for certain HTTP requests
php://output	Write-only access to the output buffer, similar to print or echo statements
php://fd/n	Direct access to file descriptor n (owned by the PHP process)
php://filter	Special wrapper allowing application of filters during stream access

```
$handle = fopen("php://stdout", 'w');
fwrite($handle, "Hi there!\n");
fclose($handle);
```

```
Output:
Hi there!
```

In truth we haven't accomplished anything that couldn't be done with a simple print statement. (print is a language construct that sends data to the output buffer; when PHP is running under the cli SAPI, the buffer is immediately flushed to the STDOUT standard stream. The code above is just skipping around the buffer.) Here's another example: